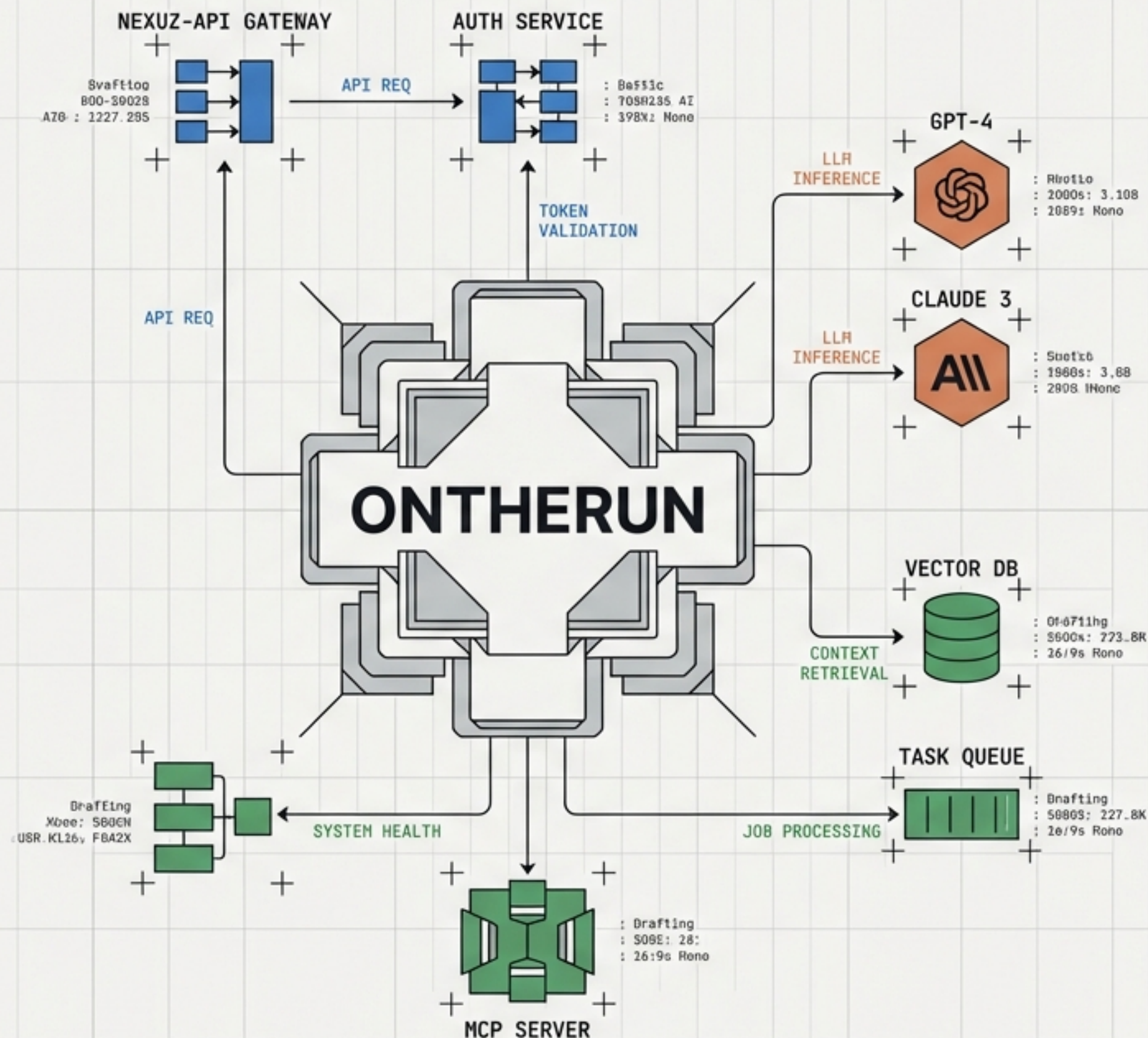


ONTHERUN

The DkZ Ecosystem Backend Backbone

Multi-Provider Integrations,
NEXUZ-API, and MCP Server
Architecture.



Powering the **DkZ** Ecosystem Through **Three Core Engines**

Model Context Protocol (MCP)

A standards-compliant server providing direct AI workspace integration with over 34 distinct tools.

NEXUZ-API

A high-speed, native REST backbone routing system state, chat, and file operations.

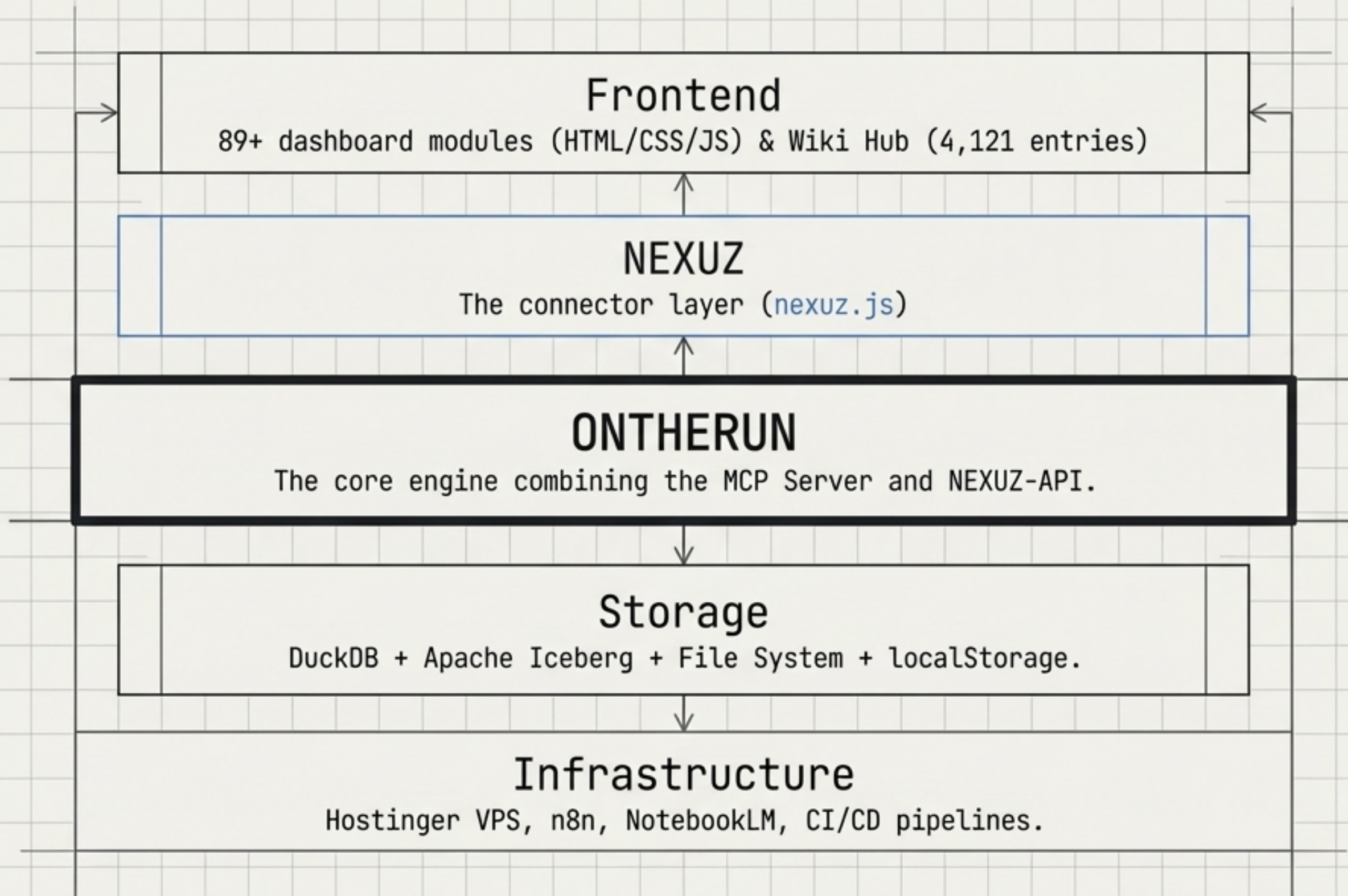
Multi-Provider Integrations

A unified gateway connecting the local environment to over 10 distinct AI and productivity platforms.

DkZ Ecosystem

Project Philosophy: Das exakte Verfahren ist wichtiger als das Ergebnis (The exact procedure is more important than the result).

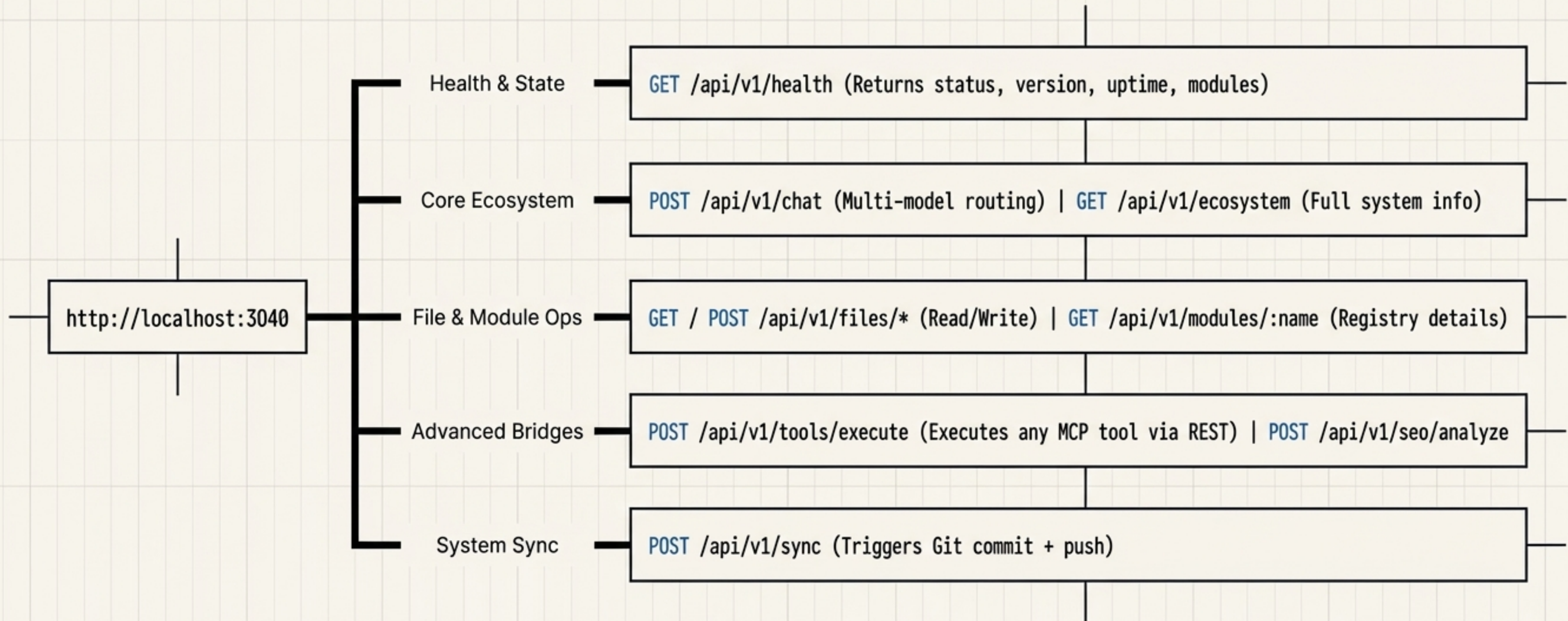
Positioning ONTHERUN as the Central Routing Layer



Two Distinct Communication Protocols for Different Ecosystem Consumers

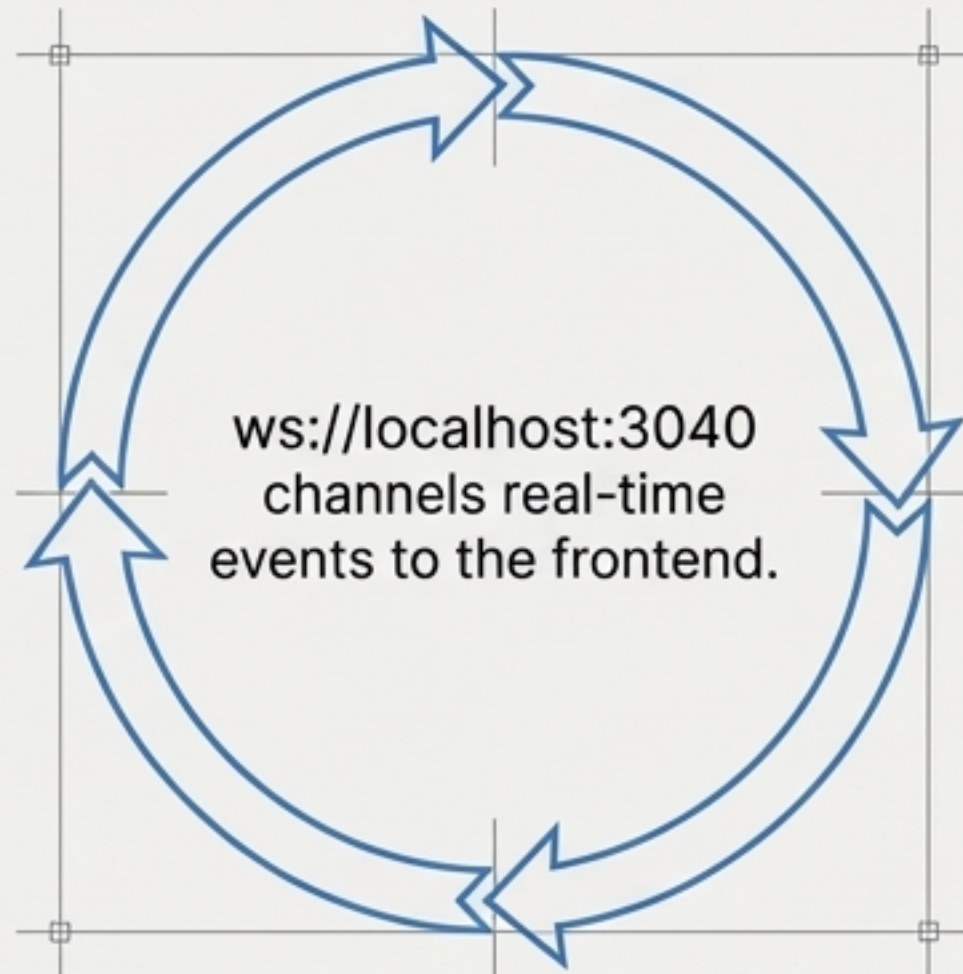
NEXUZ-API	MCP Server
Primary Consumers: Internal DkZ frontend modules and routing scripts.	Primary Consumers: External AI clients (Cursor IDE, Claude Desktop).
Architecture: Native REST communication.	Architecture: Standards-compliant Model Context Protocol.
Core Responsibilities: Managing internal system state, direct file reads, module queries, and basic chat routing.	Core Responsibilities: Allowing AI to directly interact with the system, execute complex file/workflow operations, and manage external provider integrations.

Mapping the NEXUZ-API REST Topology



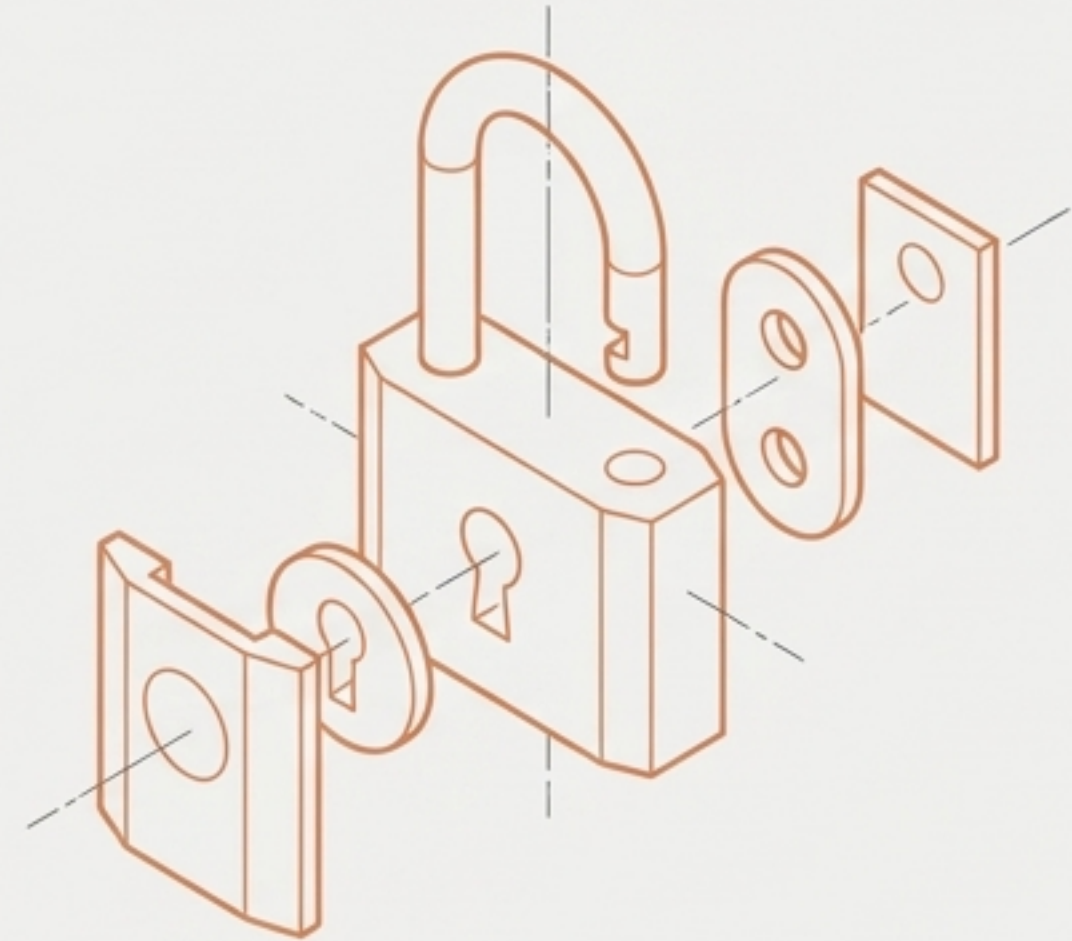
Live System Telemetry and Security Boundaries

The Telemetry Loop



- **connection:** Live connection status.
- **module:update:** Broadcasts immediate changes to ecosystem modules.
- **health:change:** Triggers alerts on system status shifts.

The Security Footprint



Current State: Gated entirely to localhost for secure, isolated local development.

Planned Architecture: Implementing an X-DKZ-Key header for future distributed API deployments.

The 34-Tool MCP Capability Matrix

AI & Content Processing

dkz_chat dkz_summarize dkz_translate
dkz_code_gen

System & File Administration

dkz_read_file dkz_write_file dkz_list_dir
dkz_ecosystem_info dkz_health

Workflow & CI/CD

dkz_workflow_create dkz_git_sync dkz_seo_analyze

External Integrations (otr_ prefix)

DevOps

otr_github_* otr_n8n_workflows otr_n8n_execute

Research

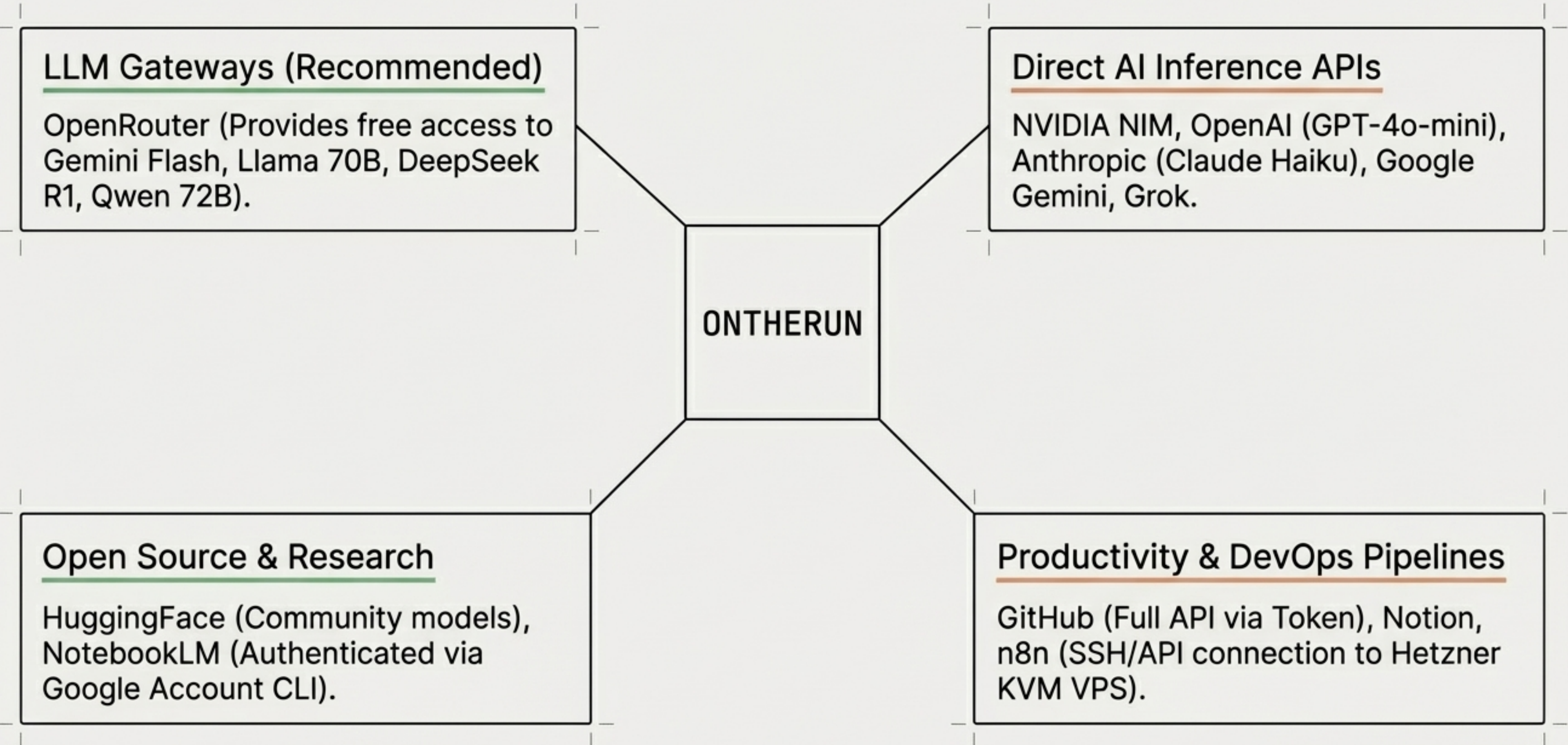
otr_notion_* otr_huggingface_* otr_notebooklm_create

Security

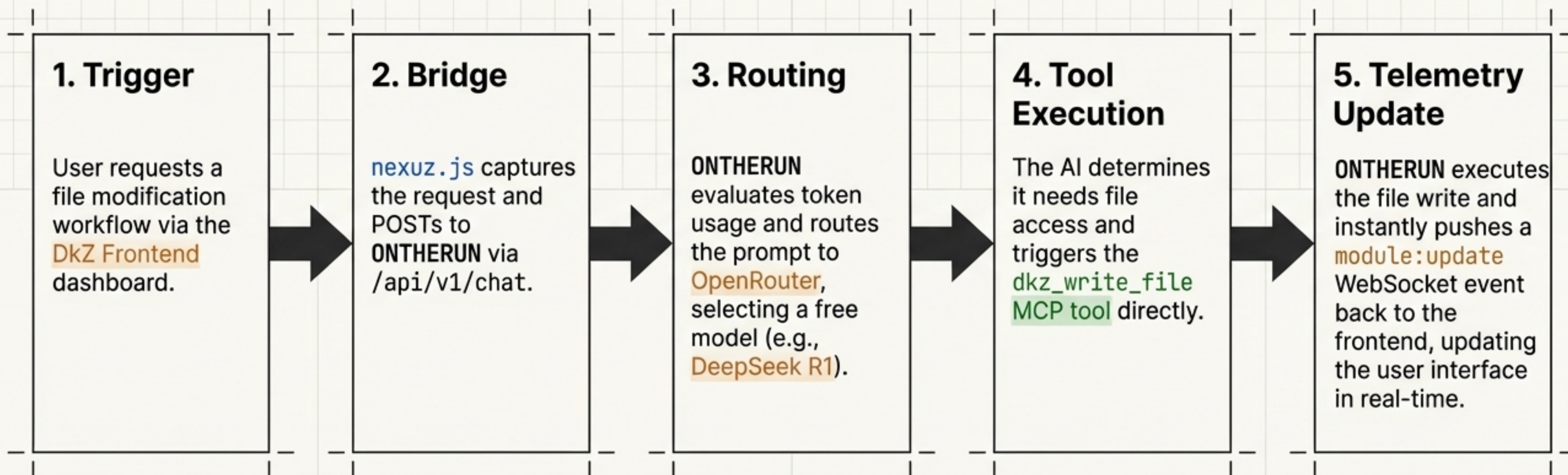
otr_token_set otr_token_status

Validation Standard: Every tool enforces strict Zod validation and adheres to precise dkz_* or otr_* naming standards.

Unified Access to the Global AI and Productivity Ecosystem



Architecture in Action: The Lifecycle of a Prompt



Frictionless Local Deployment Pathways

Prerequisites: Requires Node.js v20+ and Python 3.10+ (specifically for NotebookLM CLI).

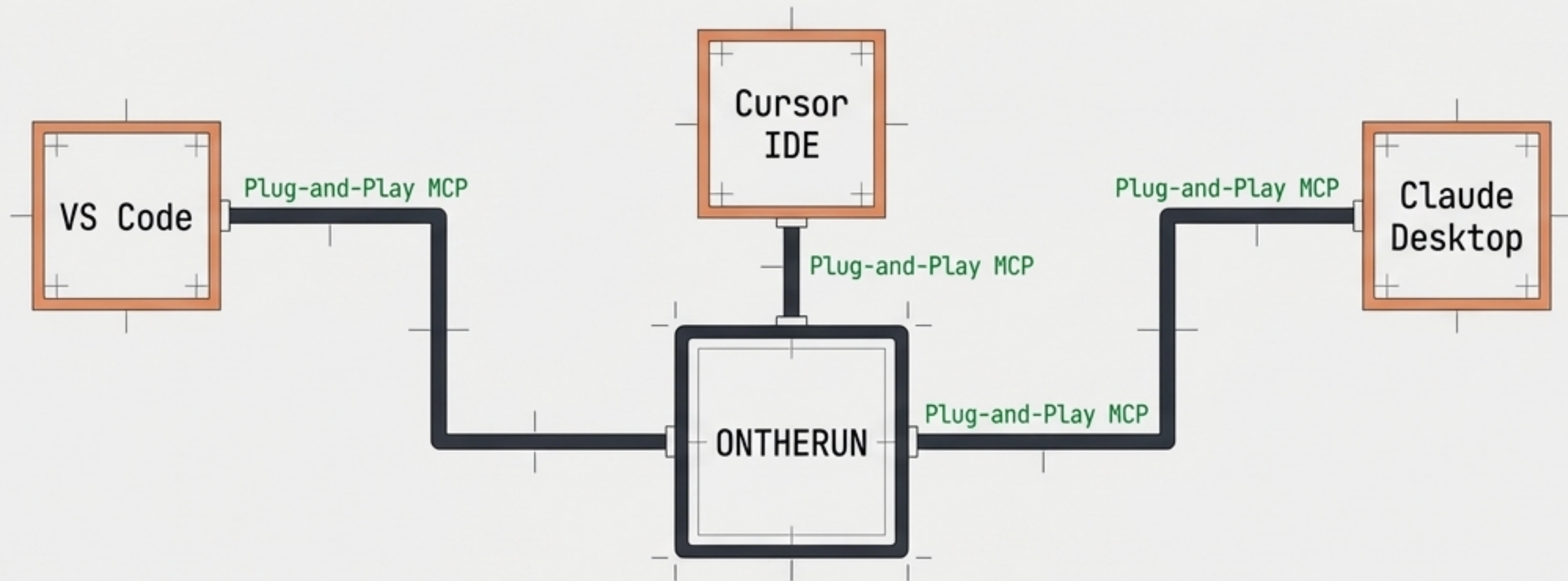
The Automated Path (One-Click)

- ❑ Launch via `DKZ_START.bat` in the DkZ root.
- ❑ Result: Automatically resolves dependencies and launches the full stack.

Configuration & Control

- ❑ API Keys and provider configurations are securely managed via `.env`.
- ❑ Alternative local overrides are handled through `config.local.json`.
- ❑ UI connection status is verifiable via the Settings Panel (Verbindungen tab).

Bringing the Ecosystem Directly into the Developer IDE



The Concept: ONTHERUN is explicitly built to interface seamlessly with top-tier AI clients, acting as a bridge between the local machine and the DkZ architecture.

Supported Environments: Drop-in MCP client configurations for VS Code, Cursor IDE, and Claude Desktop.

The Impact: Instantly grants your local AI assistant full, validated read/write/execute access to the entire DkZ ecosystem safely and locally.

Built on Open Standards and Exact Procedures

The Ecosystem Recap

1

Unified Backend Layer.

2

Communication Interfaces
(REST + MCP).

10+

Provider Platforms.

34+

Specialized Tools.

Open Source Commitment

Released under the AGPL-3.0
License.

Strict adherence to ESM
modules (import/export).

Rigorous Zod tool
validation.